Mysql的事务与锁机制分析

事务简单的操作

spring事务中NESTED就是基于savepoint实现！！

```
START TRANSACTION;    那么mysql为了减少我们键盘的消耗度，我们也可以简单的用
BEGIN;

然后我们中间写多个操作命令
INSERT INTO student(student_no,student_name,student_age,student_addr)
VALUES(5,"李四",18,"长沙");
//或者设置回滚点,回滚上面的内容
SAVEPOINT A;
UPDATE student SET student_age=student_age+1 WHERE  student_no=1;

最后，提交或者回滚
COMMIT;
ROLLBACK;
//回滚回滚点
ROLLBACK TO A;
COMMIT;
```

事务的特性

原子性（Atomicity）

COMMIT、ROLLBACK声明

刚才我们讲了，事务是要么都成功要么都失败，那么我们就得有点东西可以让2个语句要么成功，要么失败！

怎么保证多个语句能在一个原子里面去执行，其实就是开启我们的事务，然后回滚或者提交！！！

自动提交设置

那么！我们知道多个语句可以通过开启，提交或者回滚事务，那么单个语句我们是不是有事务！觉得有的刷1，没有的刷0！！！

其实每个语句都是原子的，就像它被begin START TRANSACTION开启，并且COMMIT。

```
BEGIN;
UPDATE student set student_name="灰灰老师1" where id=1;
COMMIT;
```

但是不能ROLLBACK用来撤消效果。除非在语句执行期间发生错误，则会回滚该语句。

我们的单个语句都是会自动提交的，我们可以自行去开启并关闭！！默认是开启的！！

```
show VARIABLES like 'autocommit';  查询自动开启提交
set SESSION autocommit=0;  关闭自动提交
```

咕泡出品

那么如果我们去把它关闭的话，那么我们就得去包一包了！如果不包，那么就不会提交

隐式提交

但是也不是所有的语句加上begin就会开启事务，因为有很多语句是能导致隐式提交并且无法回滚

比如：ddl语句 ，具体见 https://dev.mysql.com/doc/refman/8.0/en/implicit-commit.html

DDL(Data Definition Language) ALTER 、CREATE

DML(Data Manipulation Language) SELELCT、 INSERT

DCL（Data Control Languag） 数据控制语言 COMMIT

肯定是有1个地方能够记录我的sql语句，然后出问题了我能恢复到之前的状态！所以，mysql有个概念叫undolog，翻译过来叫做回滚日志，

持久性（Durability [dərəˈbɪlɪti)

持久性是为了保证断点等异常的情况，还能保证我们commit的数据不丢失！并且不会回滚！不会出现我commit，重启后又被回滚了！

有个undolog能保证原子性，同样的，也有1个redolog，重做日志去保证特殊情况的数据丢失！

redolog会记录每次事务的执行语句！当发生断电等比较不可控的因素后，能根据redolog进行数据恢复！！！

隔离性（Isolation [ˌaɪsəˈleɪʃn] ）

隔离性，就是需要所有的事务做到互不干扰，假如我在一个事务里面操作一个表，其他人就不能操作这个表，就像你有1张银行卡，你在取钱的同时，你老婆不能拿这张卡转账取钱！所以，大家取私房钱的时候还是得小心点，别被你老婆发现了！

这个隔离性也是我们今天要讲的重点，怎么去实现隔离性！！等下我们再细讲！！！

一致性（Consistency [kənˈsɪstənsi] ）

一致性就很好理解，就是要都能成功或者都失败，就像刚才讲的，你不能给你老婆打钱，你钱扣掉了，但是你老婆的账户钱没收到！

我们的原子性，持久性，隔离性都是为了保证我们事务的一致性！！！

事务并发产生的问题

脏读（dirty_read）

我读到了没有提交的数据 没有commit的数据

不可重复读（non-repeatable read）

我读到了重复单但结果不一样的数据

幻读（phantom）

我读到了新增的数据

隔离级别

并发问题，mysql为大家提供了不同的解决方案，不同的解决方案解决不同层次的问题！！！

那么，这个就是我们的隔离级别！！！不同的隔离级别会解决不同的问题！！！！

官网地址：https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html

那么，不同的隔离级别解决了什么问题呢？

其实sql标准92版 官方都有定义出来

sql标准不是数据库厂商定义出来的，大家不要以为sql语言是什么mysql，sqlserver搞出来的，我们发现每个数据库语句的sql语句都是差不多的，所以，sql是独立于厂商的！！ SQL是Structured Query Language的缩写，本来就属于一种查询语言！！

http://www.contrib.andrew.cmu.edu/~shadow/sql/sql1992.txt 搜索_iso

定义了四个隔离级别，右边的P1 P2 P3就是代表事务并发的3个问题，脏读，不可重复读，幻读。Possible代表在这个隔离级别下，这个问题有可能发生，换句话说，没有解决这个问题。Not Possible就是解决了这个问题。

我们可以看下官网支持了4种隔离级别，我们也可以通过sql去查询当前的隔离级别

```
SHOW GLOBAL VARIABLES LIKE '%isolation%';  //全局隔离级别
SHOW SESSION VARIABLES LIKE '%isolation%';

set SESSION autocommit=0;   //关闭自动提交
```

更改隔离级别：https://dev.mysql.com/doc/refman/8.0/en/set-transaction.html

更改语句

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;   修改当前会话的隔离级别为读未提交
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;   读已提交级别
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  RR级别
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;  串行化级别
```

我们从READ UNCOMMITTED开始！！

```
set SESSION autocommit=0;   //关闭自动提交
BEGIN;
UPDATE student SET student_age=student_age+1 WHERE student_no=1; 修改之前的数据 不提交
```

RU会话

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED; 修改为读未提交   我们发现是会出现脏读的
set SESSION autocommit=0;
BEGIN;
SELECT * FROM student where student_no=1;
COMMIT;   //能查询到还没有提交的数据，RU没有解决任何问题
```

结论：RU没有解决任何问题

RC会话

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
BEGIN;
select * FROM student  WHERE student_no=1;   RC能解决脏读问题
COMMIT;

//那么我们继续看下RC能不能解决可重复读的问题

BEGIN;
select * FROM student  WHERE 1=1;     查询结果18
                                                                                //其
他事务会话改了原有数据并且添加新数据

 BEGIN;

 UPDATE student SET student_age=student_age+1 WHERE
student_no=1;


 INSERT INTO
student(student_no,student_name,student_age,student_addr)

 VALUES(2,"huihui",18,"changsha");

 COMMIT;

select * FROM student  WHERE 1=1;     查询结果19并且有新的数据

COMMIT;
```

结论： RC解决了脏读问题，但是没有解决不可重复读跟幻读问题

会话3

```
BEGIN;
select * FROM student  WHERE 1=1;     查询结果18
                                                                                //其
他事务会话改了原有数据并且添加新数据

 BEGIN;

 UPDATE student SET student_age=student_age+1 WHERE student_no=1;


 INSERT INTO student(student_no,student_name,student_age,student_addr)

 VALUES(2,"huihui",18,"changsha");

 COMMIT;

select * FROM student  WHERE 1=1;     查询不到最新数据以及修改的数据

COMMIT;
```

结论： RR解决了可重复读的问题，同时也解决了幻读问题

怎么实现事务的隔离级别的

innoDB下RC跟RR怎么去实现隔离级别， 怎么解决脏读、不可重复读跟幻读的！！

我们知道mysql默认的隔离级别是RR，那么我们就从RR开始分析下，它到底是怎么去解决幻读的问题的！

好，现在我们不去看什么mysql它是怎么实现的，就我们自己来想，如果让我们去实现第一个事务读不到第二个事务新加的内容，我们会有哪些方案！！

首先想到的方案：事务A读取了数据后，不管你后面的事务怎么对数据再进行更改，我们都不去读最新的了，是不是就解决了我们的问题！！！！

假如我事务A跟B都操作数据ID为1的，那么我们为了A不读到B的数据，我们是不是就可以当A第一次读到数据1后，后面不管事务B怎么修改，我都不去进行读取就能解决了！！！

mysql的这种实现方式就是我们的MVCC！也就是我们的多版本控制！

MVCC（Multi-Version Concurrent Control ）多版本并发控制

大家从字面意思应该就很好理解！！！版本控制，我一次查询的时候有一个版本，后续在同一个事物里面查询的时候，我都是查询的第一个版本！！！

那么MVCC到底是怎么实现的呢？我们慢慢来去分解！！！

首先,每个事物都有1个事务ID，并且是递增的，这个大家首先要知道的，因为后面的MVCC原理都是基于它来的！！

然后我们从每行数据的隐藏列讲起：innoDB会在每行的后面都添加3个隐藏字段

DB_TRX_ID ： 表示最近一次对本记录行作修改的事务ID,每个事物都有1个事务ID，这个为最后1次在哪个事务进行修改的！！

DB_ROLL_PTR ： 回滚指针，指向当前记录行的undo log信息 （我们不读最新的数据，就是根据这个往前找数据）

DB_ROW_ID :随着新行插入而单调递增的行ID。理解：当表没有主键或唯一非空+索引时，innodb就会使用这个行ID自动产生聚簇索引。这个DB_ROW_ID跟MVCC关系不大，但是跟我们后面的锁有一定关系

时间关系，我们不一个一个讲，那么我给大家简单讲下，那么实现MVCC实现1个重要的东西，read view

~~跟undo log~~

首先，

然后我们再看下readView结构里面的几个重要的东西

trx_ids 记录了当前正在活跃的事务ID，也就是没有提交的事务ID数组集合

min_trx_id 当前正在活跃的事务ID的最小的那个事务ID

max_trx_id 下一个即将分配的事务ID，就是当前活跃的最大的事务ID+1;

知道了这些概念以后，那么，我们看下原理https://www.processon.com/view/link/5fc8f9391e08534f6cd2ac31

当前事务修改，还是会读到最新数据（一致读取）

https://dev.mysql.com/doc/refman/8.0/en/innodb-consistent-read.html

LBCC（Lock-Based Concurrent Control） 锁并发控制

数据库的锁大家应该也都听过，大家知道哪些mysql锁！

我们不一个一个讲，但是可以给大家整理下，给大家复习下，mysql锁我们要分几个不同的维度去分析

粒度：表锁 行锁 页锁

类型：共享 排它 意向共享、意向排它

用户：乐观锁 悲观锁

算法：记录锁 临键锁 间隙锁

那么我们简单给大家分析下各种锁，首先，最简单的锁类型，共享与排他!!这个平时也是大家用得最多的行级锁！加锁防止并发！

接下来共享与排他锁，我就给大家举2个例子！！

共享锁（Shared Locks）

会话1

```
show STATUS LIKE 'innodb_row_lock%'    可以查看行锁的状态

BEGIN;
SELECT * FROM student where student_no=1 LOCK in SHARE MODE;

等待会话2执行,会发现会话2修改堵塞，但是能读
COMMIT;
```

会话2

```
//堵塞
BEGIN;
UPDATE student SET student_age=student_age+1 WHERE student_no=1;
COMMIT;


//会话1 能加共享锁读
BEGIN;
SELECT * FROM student where student_no=1 LOCK in SHARE MODE;
COMMIT;
```

我们发现加上共享锁后，其他事务时不能写跟操作的，但是能加共享锁，所以共享锁也叫读锁！！

排他锁（Exclusive Locks）

另外一个行级锁就是排他锁（Exclusive Locks,它锁的加锁方式有两种，第一种是自动加排他锁，可能是同学们没有注意到的：

我们在操作数据的时候，包括增删改，都会默认加上一个排它锁。这个也是为什么说我们数据库修改本来就自带锁，你们update数据的时候不用去担心并发的原因！！

还有一种是手工加锁，我们用一个FOR UPDATE给一行数据加上一个排它锁！

所以，排他锁，我们在开发的时候是肯定会用到的！！我们也简单演示下

会话1：

```
BEGIN;
UPDATE student SET student_age=student_age+1 WHERE student_no=1;
select * FROM student  WHERE student_no=1 FOR UPDATE ;     //或者主动加锁
//不提交等待会话2执行
COMMIT;
```

会话2：

```
//在会话1不提交的情况下是会锁住的
BEGIN;
UPDATE student SET student_age=student_age+1 WHERE student_no=1;
COMMIT;

//在会话1提交之前也不能加共享锁
BEGIN;
SELECT * FROM student where student_no=1 LOCK in SHARE MODE;
COMMIT;
```

我们也可以通过show STATUS LIKE 'innodb_row_lock%' 查看行锁被锁的情况

所以，我们发现，共享锁加了后就不能排他加锁，与排他锁是互斥的！！

兼容性	X	IX	S	IS
X	不兼容	不兼容	不兼容	不兼容
IX	不兼容	兼容	不兼容	兼容
S	不兼容	不兼容	兼容	兼容
IS	不兼容	兼容	兼容	兼容

我们看似共享和排他锁住的都是行，但是其实不是锁住行，锁住的是索引！！！举例，用student_name查询，锁住的是整个表

```
BEGIN;
SELECT * FROM student where student_name='灰灰' FOR UPDATE;

//未提交，会锁死整个表，我们去查存在的数据会导致缩表
                                BEGIN;
                                SELECT * FROM student where student_no=14
for UPDATE;
                                COMMIT;

COMMIT;
```

我们发现，如果根据非索引字段查询加锁后，会把整个表给锁掉！！

记录锁（Record lock ）

如果我们用我们的索引去查的时候，我们会发现锁定的只是我们查询出来的那条数据！！！！

```
BEGIN;
select * FROM student  WHERE student_no=1 FOR UPDATE ;
//等待会话2执行
COMMIT;
```

会话2

```
BEGIN;
UPDATE student SET student_age=student_age+1 WHERE student_no=1;     //会锁住1 ，
但是不会锁住其他数据
//等待会话2执行
COMMIT;
```

我们发现，当我查询出来的数据，加锁后，是都会锁住的，那么这个就是我们的记录锁算法！！！锁住单条数据！！！！那么这个锁算法在RC跟RR的情况下都有！！！

间隙锁（Gap lock）

但是我们来看一个情况！！！

假如，我查询加锁的不是单条数据！！！我查询了不存在的范围或者数据

```
BEGIN;
select * FROM student  WHERE student_no>15 AND  student_no<23 FOR UPDATE ;  锁定
（14，23]  一个左开右闭的空间
select * FROM student  WHERE student_no<1 FOR UPDATE ;   锁定 （-∞，1]
COMMIT;
```

那么这个时候会怎么加锁呢！！！！给大家举几个例子

```
BEGIN;
select * FROM student  WHERE student_no=-1 FOR UPDATE ;   //不是范围查询时，左开右开
INSERT INTO student(student_no,student_name,student_age,student_addr)
VALUES(1,"lily",18,"上海");     锁住的时候不能加数据
但是，可以加锁不存在的数据
COMMIT;


BEGIN;
UPDATE student SET student_age=student_age+1 WHERE student_no=14;  //不锁

UPDATE student SET student_age=student_age+1 WHERE student_no=15;  //不锁

UPDATE student SET student_age=student_age+1 WHERE student_no=23;  //锁

INSERT INTO student(student_no,student_name,student_age,student_addr)
VALUES(15,"lily",18,"上海");  //15-22的添加都锁
COMMIT;
```

那么我们是不是可以整理下！！！当我们查询的记录不存在，并且索引查询的这个范围在我们的2个索引数据中间，就会把这个整个区间的insert锁住！！！这个怎么理解?

2个索引数据中间，那么我们还是以刚刚那个例子为例，我们是不是把已有的数据都分成区间，我们的查询是基于student_no！！那么我们可以分成这样的区间

https://processon.com/diagraming/5fd33090e401fd06ddbc9596

只要我的查询条件落在了这个区间，那么就会锁住整个区间！！！

比如

```
SELECT * FROM student where student_no <12 and student_no>3 FOR UPDATE;   //来告诉我，会锁住哪个区间！！(3,14)
```

但是记住，它只会在锁不存在的insert,修改删除不存在的数据是不会锁的！！！

对，那么这种锁住区间间隙的，就叫做间隙锁，那么整个锁，只有在RR的隔离级别才会存在！！！欸，我们的RR刚才讲了，innoDB它会解决幻读的问题！！那么在锁并发控制下面，就是基于间隙锁算法实现的!!!

临键锁（Next-Key Lock）

那么，老师老师，刚才讲的一个是只查询到了我表里面已有的数据，一个是我只查询我不存在的区间数据加锁，那么如果我查询的语句里，不仅仅命中了Record记录，还包含了间隙，那么它到底是怎么加锁的呢！！！！

其实，当出现这样的情况，那么简单嘛，既然你又查询到数据，又包含了区间，那么就记录锁跟间隙锁一起用呗！！！！

来，大家看例子！！

```
BEGIN;

select * FROM student  WHERE student_no>15 AND  student_no<=23 FOR UPDATE ;  锁定（14，23]和 （23,35]  中间内的数据，锁定左开右闭的区间，范围包含临界点23，所以，锁定下一个区间

select * FROM student  WHERE student_no>37 FOR UPDATE ;   锁定  （35,55] 和（55，+∞）；因为范围包含了临界点55，所以，锁定下一区间（55，+∞）

select * FROM student  WHERE student_no>15 AND  student_no<35 FOR UPDATE ;  锁定（14，23] 和（23,35]  范围没有包含临界点35，不会锁下一个左开右闭区间

select * FROM student  WHERE  student_no>=35 AND student_no<55 FOR UPDATE;  记录锁锁定35，间隙锁锁定(35,55]，不会锁定55之后的数据，没有包含临界点55
COMMIT;
```

那么，这种情况mysql里面有个专门的名词，叫做临键锁（Next-Key Lock），它是MySQL里面默认的行锁算法，相当于记录锁加上间隙锁。

唯一性索引，等值查询匹配到一条记录的时候，退化成记录锁。

没有匹配到任何记录的时候，退化成间隙锁。但是有1个点不一样！！！但是跟间隙锁有点不一样！！

当范围包含临界点的时候，则会锁定下一个左开右闭区间！！不然不会锁定下一个区间,那么这个可能也是临键锁的一个由来！！！